

✓ NGBoost + DiCE: Water Quality Classification

Pipeline V4: Median + MinMax + TANPA SMOTE + Threshold Optimization + Hyperparameter Tuning

Mengikuti strategi Al Bataineh et al. (2026) yang mencapai 86.9% pada dataset yang SAMA:

- TANPA SMOTE (SMOTE terbukti menurunkan performa di V2 dan V3)
- Median Imputation
- Min-Max Normalization [0,1]
- Hyperparameter Tuning (GridSearch/RandomizedSearch)
- Threshold Optimization (pada validation set)
- SHAP Feature Analysis

Justifikasi Tidak Menggunakan SMOTE:

- V2 (dengan SMOTE): Accuracy TURUN dari 0.67 ke 0.63
- V3 (SMOTE + Threshold): Accuracy TURUN ke 0.52
- Al Bataineh et al. (2026) mencapai 86.9% TANPA SMOTE pada dataset yang sama
- Rasio imbalance 61:39 tergolong MILD - tidak memerlukan resampling agresif

Referensi Pendukung:

- Al Bataineh et al. (2026) - Hybrid ML Framework for WQI Prediction (86.9% tanpa SMOTE)
- [7] Grinsztajn et al. 2022 - Tree-based models for tabular data
- [11] Zhang et al. - Threshold moving approaches
- [13] van de Mortel & van Wingen 2025 - Data leakage prevention
- [14] Chen & Guestrin 2016 - XGBoost hyperparameter importance

✓ Cell 1: Instalasi Library

```
!pip install ngboost dice-ml xgboost scikit-learn pandas numpy matplotlib seaborn shap
```

✓ Cell 2: Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import os

# Install ngboost if not already installed
!pip install ngboost

# Sklearn
from sklearn.model_selection import (
    train_test_split, StratifiedKFold, GridSearchCV, RandomizedSearchCV
)
from sklearn.preprocessing import MinMaxScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report,
    log_loss, brier_score_loss
)
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.calibration import calibration_curve

# NGBoost
```

```

from ngboost import NGBClassifier
from ngboost.distns import Bernoulli

# XGBoost
from xgboost import XGBClassifier

# SHAP
import shap

warnings.filterwarnings("ignore")
sns.set_style("whitegrid")

# Buat folder figures jika belum ada
os.makedirs("figures", exist_ok=True)

print("All libraries imported successfully!")

```

```

Collecting ngboost
  Downloading ngboost-0.5.10-py3-none-any.whl.metadata (4.3 kB)
Collecting lifelines>=0.25 (from ngboost)
  Downloading lifelines-0.30.3-py3-none-any.whl.metadata (3.5 kB)
Requirement already satisfied: matplotlib>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from ngboost) (3.
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.12/dist-packages (from ngboost) (2.0.2)
Requirement already satisfied: scikit-learn<2.0,>=1.6 in /usr/local/lib/python3.12/dist-packages (from ngboost)
Requirement already satisfied: scipy>=1.7.2 in /usr/local/lib/python3.12/dist-packages (from ngboost) (1.16.3)
Requirement already satisfied: sympy>=1.12 in /usr/local/lib/python3.12/dist-packages (from ngboost) (1.14.0)
Requirement already satisfied: tqdm>=4.3 in /usr/local/lib/python3.12/dist-packages (from ngboost) (4.67.3)
Requirement already satisfied: pandas<3.0,>=2.1 in /usr/local/lib/python3.12/dist-packages (from lifelines>=0.2
Requirement already satisfied: autograd>=1.5 in /usr/local/lib/python3.12/dist-packages (from lifelines>=0.25->
Collecting autograd-gamma>=0.3 (from lifelines>=0.25-&gtngboost)
  Downloading autograd-gamma-0.5.0.tar.gz (4.0 kB)
  Preparing metadata (setup.py) ... done
Collecting formulaic>=0.2.2 (from lifelines>=0.25-&gtngboost)
  Downloading formulaic-1.2.2-py3-none-any.whl.metadata (7.0 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.1
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->n
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn<2.0,
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-lea
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.12-
Collecting interface-meta>=1.2.0 (from formulaic>=0.2.2->lifelines>=0.25-&gtngboost)
  Downloading interface_meta-2.0.1-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2
Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/python3.12/dist-packages (from formul
Requirement already satisfied: wrapt>=1.0 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2->li
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=2.1->
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=2.1
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->
Downloading ngboost-0.5.10-py3-none-any.whl (52 kB)
----- 52.0/52.0 kB 2.2 MB/s eta 0:00:00
Downloading lifelines-0.30.3-py3-none-any.whl (409 kB)
----- 409.1/409.1 kB 10.6 MB/s eta 0:00:00
Downloading formulaic-1.2.2-py3-none-any.whl (118 kB)
----- 118.9/118.9 kB 8.7 MB/s eta 0:00:00
Downloading interface_meta-2.0.1-py3-none-any.whl (15 kB)
Building wheels for collected packages: autograd-gamma
  Building wheel for autograd-gamma (setup.py) ... done
  Created wheel for autograd-gamma: filename=autograd_gamma-0.5.0-py3-none-any.whl size=4030 sha256=a744851d309
  Stored in directory: /root/.cache/pip/wheels/50/37/21/0a719b9d89c635e89ff24bd93b862882ad675279552013b2fb
Successfully built autograd-gamma
Installing collected packages: interface-meta, autograd-gamma, formulaic, lifelines, ngboost
Successfully installed autograd-gamma-0.5.0 formulaic-1.2.2 interface-meta-2.0.1 lifelines-0.30.3 ngboost-0.5.1
All libraries imported successfully!

```

✓ Cell 3: Load Dataset + EDA

```

import os

# Load dataset
CSV_PATH = "water_potability.csv"
RAW_URL = (
    "https://raw.githubusercontent.com/"

```

```

"aflahzaki/MetodPen-ICoICT-/main/water_potability.csv"
)

def muat_dataset(path: str = CSV_PATH, url: str = RAW_URL) -> pd.DataFrame:
    '''Memuat dataset: lokal -> unduh GitHub -> unggah manual (Colab).'''
    # (a) berkas lokal
    if os.path.exists(path):
        print(f"Memuat dataset lokal: {path}")
        return pd.read_csv(path)
    # (b) unduh otomatis dari GitHub
    try:
        import urllib.request
        print(f"Mengunduh dataset dari GitHub...\n{url}")
        urllib.request.urlretrieve(url, path)
        print("Unduhan berhasil.")
        return pd.read_csv(path)
    except Exception as exc:
        print(f"Unduhan otomatis gagal ({exc}).")
    # (c) fallback unggah manual (khusus Colab)
    try:
        from google.colab import files
        print("Silakan unggah berkas 'water_potability.csv' secara manual:")
        uploaded = files.upload()
        nama = next(iter(uploaded))
        return pd.read_csv(nama)
    except Exception as exc:
        raise FileNotFoundError(
            "Tidak dapat memuat dataset. Sediakan 'water_potability.csv'."
        ) from exc

# Memanggil fungsi untuk mendefinisikan df
df = muat_dataset()

print("=" * 60)
print("DATASET INFORMATION")
print("=" * 60)
print(f"Shape: {df.shape}")
print(f"\nFirst 5 rows:")
display(df.head())

print(f"\nDataset Info:")
df.info()

print(f"\nMissing Values:")
print(df.isnull().sum())

print(f"\nDistribusi Kelas (Potability):")
print(df["Potability"].value_counts())
print(f"\nPersentase:")
print(df["Potability"].value_counts(normalize=True) * 100)

```

Memuat dataset lokal: water_potability.csv

DATASET INFORMATION

Shape: (3276, 10)

First 5 rows:

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	
0	NaN	204.890455	20791.318981	7.300212	368.516441	564.308654	10.379783	86.990970	
1	3.716080	129.422921	18630.057858	6.635246	NaN	592.885359	15.180013	56.329076	
2	8.099124	224.236259	19909.541732	9.275884	NaN	418.606213	16.868637	66.420093	
3	8.316766	214.373394	22018.417441	8.059332	356.886136	363.266516	18.436524	100.341674	
4	9.092223	181.101509	17978.986339	6.546600	310.135738	398.410813	11.558279	31.997993	

Dataset Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 3276 entries, 0 to 3275

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	ph	2785 non-null	float64
1	Hardness	3276 non-null	float64
2	Solids	3276 non-null	float64
3	Chloramines	3276 non-null	float64
4	Sulfate	2495 non-null	float64
5	Conductivity	3276 non-null	float64
6	Organic_carbon	3276 non-null	float64
7	Trihalomethanes	3114 non-null	float64
8	Turbidity	3276 non-null	float64
9	Potability	3276 non-null	int64

dtypes: float64(9), int64(1)

memory usage: 256.1 KB

Missing Values:

ph	491
Hardness	0
Solids	0
Chloramines	0
Sulfate	781
Conductivity	0
Organic_carbon	0
Trihalomethanes	162
Turbidity	0
Potability	0

dtype: int64

Distribusi Kelas (Potability):

Potability

0 1998

1 1278

Name: count, dtype: int64

Persentase:

Potability

0 60.989011

1 39.010989

Name: proportion, dtype: float64

✓ Cell 4: Preprocessing (Ikuti Al Bataineh)

Menggunakan **Median Imputation** untuk mengisi missing values, sama seperti Al Bataineh et al. (2026).

```
# Pisahkan features dan target
X = df.drop("Potability", axis=1)
y = df["Potability"]

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(f"Missing values SEBELUM imputation:")
print(X.isnull().sum())

# 1. Median Imputation (sama dengan Al Bataineh)
```

```
# 1. Median Imputation (sama dengan AZ datasetnya)
imputer = SimpleImputer(strategy="median")
X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)

# 2. Verifikasi tidak ada missing values
print(f"Missing values SETELAH imputation: {X_imputed.isnull().sum().sum()}")
print("Median values used for imputation:")
print(pd.Series(imputer.statistics_, index=X.columns))
```

```
Features shape: (3276, 9)
Target shape: (3276,)
Missing values SEBELUM imputation:
ph                491
Hardness          0
Solids            0
Chloramines       0
Sulfate           781
Conductivity      0
Organic_carbon    0
Trihalomethanes   162
Turbidity         0
dtype: int64
Missing values SETELAH imputation: 0
Median values used for imputation:
ph                7.036752
Hardness          196.967627
Solids            20927.833607
Chloramines       7.130299
Sulfate           333.073546
Conductivity      421.884968
Organic_carbon    14.218338
Trihalomethanes   66.622485
Turbidity         3.955028
dtype: float64
```

✓ Cell 5: Stratified Split 70/15/15

Split dataset menjadi 70% training, 15% validation, 15% test.

```
# Split pertama: 85% temp, 15% test (stratified)
X_temp, X_test, y_temp, y_test = train_test_split(
    X_imputed, y, test_size=0.15, stratify=y, random_state=42
)

# Split kedua: dari temp, 15/85 ~ 0.1765 sebagai validation
val_ratio = 0.15 / 0.85 # ~0.1765
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=val_ratio, stratify=y_temp, random_state=42
)

print("Dataset Split (70/15/15):")
print(f"  Training:  {X_train.shape[0]} samples ({X_train.shape[0]/len(X_imputed)*100:.1f}%)")
print(f"  Validation: {X_val.shape[0]} samples ({X_val.shape[0]/len(X_imputed)*100:.1f}%)")
print(f"  Test:       {X_test.shape[0]} samples ({X_test.shape[0]/len(X_imputed)*100:.1f}%)")
print("\nDistribusi kelas per split:")
print(f"  Train - Class 0: {(y_train==0).sum()} ({(y_train==0).mean()*100:.1f}%), Class 1: {(y_train==1).sum()}")
print(f"  Val   - Class 0: {(y_val==0).sum()} ({(y_val==0).mean()*100:.1f}%), Class 1: {(y_val==1).sum()} ({(y_val==1).mean()*100:.1f}%)")
print(f"  Test  - Class 0: {(y_test==0).sum()} ({(y_test==0).mean()*100:.1f}%), Class 1: {(y_test==1).sum()} ({(y_test==1).mean()*100:.1f}%)")
```

```
Dataset Split (70/15/15):
  Training:  2292 samples (70.0%)
  Validation: 492 samples (15.0%)
  Test:       492 samples (15.0%)
```

```
Distribusi kelas per split:
  Train - Class 0: 1398 (61.0%), Class 1: 894 (39.0%)
  Val   - Class 0: 300 (61.0%), Class 1: 192 (39.0%)
  Test  - Class 0: 300 (61.0%), Class 1: 192 (39.0%)
```

✓ Cell 6: Min-Max Scaling (Fit on Train Only)

Min-Max Normalization [0,1] - fit hanya pada training data untuk mencegah data leakage.

```
# Min-Max Scaling - fit on train only
```

```

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print("Min-Max Scaling applied (fit on training data only).")
print("\nTraining data range:")
print(f"  Min: {X_train_scaled.min(axis=0)}")
print(f"  Max: {X_train_scaled.max(axis=0)}")

```

Min-Max Scaling applied (fit on training data only).

Training data range:
 Min: [0. 0. 0. 0. 0. 0. 0. 0. 0.]
 Max: [1. 1. 1. 1. 1. 1. 1. 1. 1.]

✓ Cell 7: Feature Selection via SHAP (Ikuti Al Bataineh)

Al Bataineh menggunakan XGBoost + SHAP untuk ranking fitur. Kita adaptasi dengan:

1. Train preliminary XGBoost pada training data
2. Compute SHAP values
3. Rank fitur berdasarkan mean |SHAP|
4. Evaluasi ALL features vs subsets

```

# Train preliminary XGBoost untuk SHAP analysis
preliminary_xgb = XGBClassifier(
    n_estimators=100, random_state=42, eval_metric="logloss", verbosity=0
)
preliminary_xgb.fit(X_train_scaled, y_train)

# Compute SHAP values
explainer = shap.TreeExplainer(preliminary_xgb)
shap_values = explainer.shap_values(X_train_scaled)

# Rank features
feature_names = X.columns.tolist()
mean_shap = np.abs(shap_values).mean(axis=0)
feature_ranking = pd.DataFrame({
    "Feature": feature_names,
    "Mean |SHAP|": mean_shap
}).sort_values("Mean |SHAP|", ascending=False)

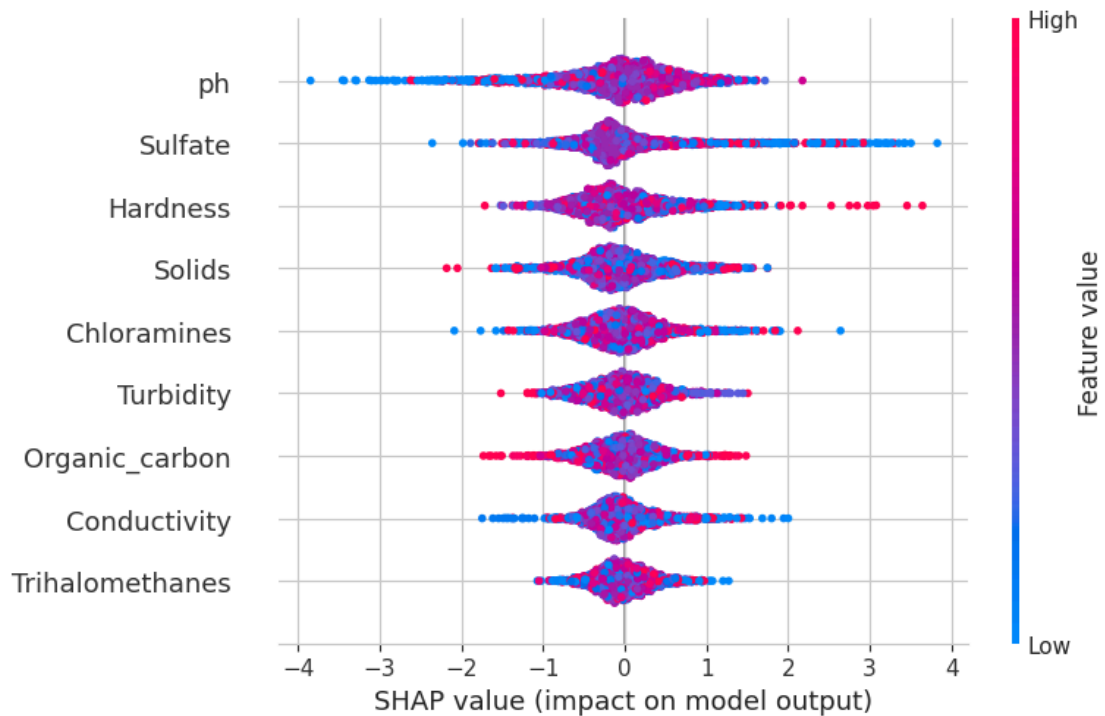
print("Feature Ranking (SHAP):")
print(feature_ranking.to_string(index=False))

# Visualisasi SHAP summary
shap.summary_plot(shap_values, X_train_scaled, feature_names=feature_names, show=False)
plt.tight_layout()
plt.savefig("figures/shap_feature_importance_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()

```

Feature Ranking (SHAP):

Feature	Mean SHAP
ph	0.515183
Sulfate	0.460963
Hardness	0.406638
Solids	0.370910
Chloramines	0.333251
Turbidity	0.302262
Organic_carbon	0.290542
Conductivity	0.284584
Trihalomethanes	0.245287



✓ Evaluasi Feature Subsets

```
# Evaluasi: ALL features vs TOP-7 vs TOP-5
sorted_features = feature_ranking["Feature"].tolist()

subsets = {
    "ALL (9 features)": list(range(len(feature_names))),
    "TOP-7 features": [feature_names.index(f) for f in sorted_features[:7]],
    "TOP-5 features": [feature_names.index(f) for f in sorted_features[:5]],
}

print("Feature Subset Evaluation (XGBoost, Validation AUC):")
print("-" * 50)

best_subset_name = None
best_subset_auc = 0
best_subset_indices = None

for name, indices in subsets.items():
    xgb_temp = XGBClassifier(n_estimators=100, random_state=42, eval_metric="logloss", verbosity=0)
    xgb_temp.fit(X_train_scaled[:, indices], y_train)
    val_prob = xgb_temp.predict_proba(X_val_scaled[:, indices])[:, 1]
    auc = roc_auc_score(y_val, val_prob)
    features_used = [feature_names[i] for i in indices]
    print(f" {name}: AUC = {auc:.4f} | Features: {features_used}")
    if auc > best_subset_auc:
        best_subset_auc = auc
        best_subset_name = name
        best_subset_indices = indices

print(f"\nBest subset: {best_subset_name} (AUC: {best_subset_auc:.4f})")
print("\nNote: Menggunakan ALL features untuk pipeline selanjutnya")
print("(Feature selection hanya untuk analisis, bukan removal, kecuali ada perbedaan signifikan)")

# Gunakan semua fitur (konsisten dengan Al Bataineh yang menggunakan semua fitur)
```

```
selected_feature_indices = list(range(len(feature_names)))
selected_features = feature_names
print(f"\nFinal selected features: {selected_features}")
```

Feature Subset Evaluation (XGBoost, Validation AUC):

```
-----
ALL (9 features): AUC = 0.6429 | Features: ['ph', 'Hardness', 'Solids', 'Chloramines', 'Sulfate', 'Conductivi
TOP-7 features: AUC = 0.6612 | Features: ['ph', 'Sulfate', 'Hardness', 'Solids', 'Chloramines', 'Turbidity',
TOP-5 features: AUC = 0.6620 | Features: ['ph', 'Sulfate', 'Hardness', 'Solids', 'Chloramines']
```

Best subset: TOP-5 features (AUC: 0.6620)

Note: Menggunakan ALL features untuk pipeline selanjutnya
(Feature selection hanya untuk analisis, bukan removal, kecuali ada perbedaan signifikan)

Final selected features: ['ph', 'Hardness', 'Solids', 'Chloramines', 'Sulfate', 'Conductivity', 'Organic_carbon']

✓ Cell 8: Hyperparameter Tuning (TANPA SMOTE)

Tuning menggunakan:

- NGBoost: Manual loop (tidak support GridSearchCV)
- XGBoost: RandomizedSearchCV (50 iterations)
- Random Forest: RandomizedSearchCV (50 iterations)

```
cv_inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
```

```
# --- NGBoost Tuning ---
```

```
# NGBoost tidak support GridSearchCV langsung, jadi manual loop:
```

```
print("="*60)
```

```
print("NGBoost Hyperparameter Tuning (Manual Search)")
```

```
print("="*60)
```

```
best_ngb_score = 0
```

```
best_ngb_params = {}
```

```
ngb_results = []
```

```
for n_est in [200, 300, 500]:
```

```
    for lr in [0.01, 0.05, 0.1]:
```

```
        for mb in [0.8, 1.0]:
```

```
            ngb_temp = NGBClassifier(
```

```
                Dist=Bernoulli,
```

```
                n_estimators=n_est,
```

```
                learning_rate=lr,
```

```
                minibatch_frac=mb,
```

```
                col_sample=0.8,
```

```
                random_state=42,
```

```
                verbose=False
```

```
            )
```

```
            ngb_temp.fit(X_train_scaled, y_train)
```

```
            val_prob = ngb_temp.predict_proba(X_val_scaled)[: , 1]
```

```
            val_auc = roc_auc_score(y_val, val_prob)
```

```
            ngb_results.append({
```

```
                "n_estimators": n_est, "learning_rate": lr,
```

```
                "minibatch_frac": mb, "val_auc": val_auc
```

```
            })
```

```
            if val_auc > best_ngb_score:
```

```
                best_ngb_score = val_auc
```

```
                best_ngb_params = {
```

```
                    "n_estimators": n_est,
```

```
                    "learning_rate": lr,
```

```
                    "minibatch_frac": mb
```

```
                }
```

```
print(f"Best NGBoost params: {best_ngb_params}")
```

```
print(f"Best NGBoost Val AUC: {best_ngb_score:.4f}")
```

```
print(f"Total combinations evaluated: {len(ngb_results)}")
```

```
=====
NGBoost Hyperparameter Tuning (Manual Search)
```

```
=====
Best NGBoost params: {'n_estimators': 300, 'learning_rate': 0.1, 'minibatch_frac': 0.8}
```

Best NGBoost Val AUC: 0.6489
Total combinations evaluated: 18

```
# --- XGBoost Tuning ---
print("="*60)
print("XGBoost Hyperparameter Tuning (RandomizedSearchCV)")
print("="*60)

xgb_param_grid = {
    "max_depth": [3, 4, 5, 6],
    "learning_rate": [0.01, 0.05, 0.1],
    "n_estimators": [200, 300, 500],
    "subsample": [0.7, 0.8, 0.9],
    "colsample_bytree": [0.7, 0.8, 0.9],
    "min_child_weight": [1, 3, 5],
    "gamma": [0, 0.1, 0.2],
    "reg_alpha": [0, 0.01, 0.1],
    "reg_lambda": [1, 1.5, 2]
}

xgb_search = RandomizedSearchCV(
    XGBClassifier(eval_metric="logloss", use_label_encoder=False, verbosity=0, random_state=42),
    xgb_param_grid,
    n_iter=50,
    scoring="roc_auc",
    cv=cv_inner,
    random_state=42,
    n_jobs=-1,
    verbose=0
)
xgb_search.fit(X_train_scaled, y_train)
print(f"Best XGBoost params: {xgb_search.best_params_}")
print(f"Best XGBoost CV AUC: {xgb_search.best_score_:.4f}")
```

```
=====
XGBoost Hyperparameter Tuning (RandomizedSearchCV)
=====
Best XGBoost params: {'subsample': 0.8, 'reg_lambda': 2, 'reg_alpha': 0, 'n_estimators': 500, 'min_child_weight': 1}
Best XGBoost CV AUC: 0.6764
```

```
# --- Random Forest Tuning ---
print("="*60)
print("Random Forest Hyperparameter Tuning (RandomizedSearchCV)")
print("="*60)

rf_param_grid = {
    "n_estimators": [200, 300, 500],
    "max_depth": [5, 8, 10, 15, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", None]
}

rf_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    rf_param_grid,
    n_iter=50,
    scoring="roc_auc",
    cv=cv_inner,
    random_state=42,
    n_jobs=-1,
    verbose=0
)
rf_search.fit(X_train_scaled, y_train)
print(f"Best RF params: {rf_search.best_params_}")
print(f"Best RF CV AUC: {rf_search.best_score_:.4f}")
```

```
=====
Random Forest Hyperparameter Tuning (RandomizedSearchCV)
=====
Best RF params: {'n_estimators': 500, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': None, 'max_depth': 15}
Best RF CV AUC: 0.6828
```

Cell 9: Train Final Models dengan Best Parameters

```
ngb_final = NGBClassifier(  
    Dist=Bernoulli,  
    n_estimators=best_ngb_params["n_estimators"],  
    learning_rate=best_ngb_params["learning_rate"],  
    minibatch_frac=best_ngb_params["minibatch_frac"],  
    col_sample=0.8,  
    random_state=42,  
    verbose=False  
)  
ngb_final.fit(X_train_scaled, y_train, X_val=X_val_scaled, Y_val=y_val)  
  
# XGBoost with best params  
xgb_final = xgb_search.best_estimator_  
  
# Random Forest with best params  
rf_final = rf_search.best_estimator_  
  
print("Final models trained successfully!")  
print(f"\nNGBoost params: {best_ngb_params}")  
print(f"XGBoost params: {xgb_search.best_params}")  
print(f"RF params: {rf_search.best_params}")
```

Final models trained successfully!

NGBoost params: {'n_estimators': 300, 'learning_rate': 0.1, 'minibatch_frac': 0.8}
XGBoost params: {'subsample': 0.8, 'reg_lambda': 2, 'reg_alpha': 0, 'n_estimators': 500, 'min_child_weight': 1,
RF params: {'n_estimators': 500, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': None, 'max_dept

Cell 10: Threshold Optimization (pada Validation Set)

Mencari threshold optimal yang maximize F1-Score per model. Range dibatasi 0.30-0.70 untuk menghindari extreme thresholds (lesson learned dari V3).

```
def find_optimal_threshold(y_true, y_prob):  
    """Find optimal threshold that maximizes F1-score on validation set."""  
    thresholds = np.arange(0.30, 0.70, 0.01)  
    best_threshold = 0.5  
    best_f1 = 0  
    for t in thresholds:  
        y_pred = (y_prob >= t).astype(int)  
        f1 = f1_score(y_true, y_pred, zero_division=0)  
        if f1 > best_f1:  
            best_f1 = f1  
            best_threshold = t  
    return best_threshold, best_f1  
  
# Get validation probabilities  
ngb_val_prob = ngb_final.predict_proba(X_val_scaled)[: , 1]  
xgb_val_prob = xgb_final.predict_proba(X_val_scaled)[: , 1]  
rf_val_prob = rf_final.predict_proba(X_val_scaled)[: , 1]  
  
# Find optimal thresholds  
ngb_opt_thresh, ngb_opt_f1 = find_optimal_threshold(y_val, ngb_val_prob)  
xgb_opt_thresh, xgb_opt_f1 = find_optimal_threshold(y_val, xgb_val_prob)  
rf_opt_thresh, rf_opt_f1 = find_optimal_threshold(y_val, rf_val_prob)  
  
print("Optimal Thresholds (Validation Set):")  
print(f"  NGBoost: {ngb_opt_thresh:.2f} (F1: {ngb_opt_f1:.4f})")  
print(f"  XGBoost: {xgb_opt_thresh:.2f} (F1: {xgb_opt_f1:.4f})")  
print(f"  Random Forest: {rf_opt_thresh:.2f} (F1: {rf_opt_f1:.4f})")
```

Optimal Thresholds (Validation Set):
NGBoost: 0.30 (F1: 0.5418)
XGBoost: 0.38 (F1: 0.5831)
Random Forest: 0.31 (F1: 0.5895)

✓ Cell 11: Evaluasi pada Test Set

Evaluasi dengan 2 threshold: default 0.5 DAN optimal threshold. Metrics: Accuracy, Precision, Recall, F1, AUC-ROC, NLL, ECE.

```
def compute_ece(y_true, y_prob, n_bins=10):
    """Compute Expected Calibration Error."""
    bin_boundaries = np.linspace(0, 1, n_bins + 1)
    ece = 0.0
    for i in range(n_bins):
        mask = (y_prob >= bin_boundaries[i]) & (y_prob < bin_boundaries[i+1])
        if mask.sum() > 0:
            bin_acc = y_true[mask].mean()
            bin_conf = y_prob[mask].mean()
            ece += mask.sum() * np.abs(bin_acc - bin_conf)
    return ece / len(y_true)

def evaluate_model(y_true, y_prob, threshold, model_name):
    """Evaluate model with given threshold."""
    y_pred = (y_prob >= threshold).astype(int)
    return {
        "Model": model_name,
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1-Score": f1_score(y_true, y_pred, zero_division=0),
        "AUC-ROC": roc_auc_score(y_true, y_prob),
        "NLL": log_loss(y_true, y_prob),
        "ECE": compute_ece(y_true, y_prob),
    }

# Get test probabilities
ngb_test_prob = ngb_final.predict_proba(X_test_scaled)[: , 1]
xgb_test_prob = xgb_final.predict_proba(X_test_scaled)[: , 1]
rf_test_prob = rf_final.predict_proba(X_test_scaled)[: , 1]

# Evaluate with both thresholds
results = []
results.append(evaluate_model(y_test, ngb_test_prob, 0.5, "NGBoost(0.5)"))
results.append(evaluate_model(y_test, ngb_test_prob, ngb_opt_thresh, f"NGBoost({ngb_opt_thresh:.2f})"))
results.append(evaluate_model(y_test, xgb_test_prob, 0.5, "XGBoost(0.5)"))
results.append(evaluate_model(y_test, xgb_test_prob, xgb_opt_thresh, f"XGBoost({xgb_opt_thresh:.2f})"))
results.append(evaluate_model(y_test, rf_test_prob, 0.5, "RF(0.5)"))
results.append(evaluate_model(y_test, rf_test_prob, rf_opt_thresh, f"RF({rf_opt_thresh:.2f})"))

results_df = pd.DataFrame(results)
print("="*80)
print("TEST SET EVALUATION RESULTS")
print("="*80)
print(results_df.to_string(index=False))

# Confusion matrices
print("\n" + "="*80)
print("CONFUSION MATRICES (Test Set)")
print("="*80)
for name, prob, thresh in [("NGBoost", ngb_test_prob, ngb_opt_thresh),
                           ("XGBoost", xgb_test_prob, xgb_opt_thresh),
                           ("RF", rf_test_prob, rf_opt_thresh)]:
    y_pred_default = (prob >= 0.5).astype(int)
    y_pred_opt = (prob >= thresh).astype(int)
    print(f"\n{name} (threshold=0.5):")
    print(confusion_matrix(y_test, y_pred_default))
    print(f"\n{name} (threshold={thresh:.2f}):")
    print(confusion_matrix(y_test, y_pred_opt))
```

=====

TEST SET EVALUATION RESULTS

=====

Model	Threshold	Accuracy	Precision	Recall	F1-Score	AUC-ROC	NLL	ECE
NGBoost(0.5)	0.50	0.662602	0.638298	0.312500	0.419580	0.639913	0.785004	0.110416
NGBoost(0.30)	0.30	0.579268	0.469388	0.598958	0.526316	0.639913	0.785004	0.110416
XGBoost(0.5)	0.50	0.693089	0.735632	0.333333	0.458781	0.678924	0.609666	0.047657

XGBoost(0.38)	0.38	0.630081	0.522936	0.593750	0.556098	0.678924	0.609666	0.047657
RF(0.5)	0.50	0.686992	0.779412	0.276042	0.407692	0.689896	0.604950	0.044312
RF(0.31)	0.31	0.528455	0.446237	0.864583	0.588652	0.689896	0.604950	0.044312

=====

CONFUSION MATRICES (Test Set)

=====

NGBost (threshold=0.5):

```
[[266  34]
 [132  60]]
```

NGBost (threshold=0.30):

```
[[170 130]
 [ 77 115]]
```

XGBoost (threshold=0.5):

```
[[277  23]
 [128  64]]
```

XGBoost (threshold=0.38):

```
[[196 104]
 [ 78 114]]
```

RF (threshold=0.5):

```
[[285  15]
 [139  53]]
```

RF (threshold=0.31):

```
[[ 94 206]
 [ 26 166]]
```

✓ Cell 12: McNemar's Test

Perbandingan statistik antar model menggunakan McNemar's test. Menggunakan prediksi dengan threshold yang memberikan accuracy terbaik per model.

```
from scipy.stats import chi2

def mcnemar_test(y_true, y_pred1, y_pred2):
    """Perform McNemar's test between two classifiers."""
    # Build contingency table
    # b = pred1 correct, pred2 wrong
    # c = pred1 wrong, pred2 correct
    correct1 = (y_pred1 == y_true)
    correct2 = (y_pred2 == y_true)
    b = ((correct1) & (~correct2)).sum()
    c = ((~correct1) & (correct2)).sum()

    # McNemar's statistic with continuity correction
    if b + c == 0:
        return 0.0, 1.0
    statistic = (abs(b - c) - 1)**2 / (b + c)
    p_value = 1 - chi2.cdf(statistic, df=1)
    return statistic, p_value

# Determine best threshold per model (by accuracy on test set)
ngb_pred_default = (ngb_test_prob >= 0.5).astype(int)
ngb_pred_opt = (ngb_test_prob >= ngb_opt_thresh).astype(int)
ngb_best_pred = ngb_pred_opt if accuracy_score(y_test, ngb_pred_opt) >= accuracy_score(y_test, ngb_pred_default)

xgb_pred_default = (xgb_test_prob >= 0.5).astype(int)
xgb_pred_opt = (xgb_test_prob >= xgb_opt_thresh).astype(int)
xgb_best_pred = xgb_pred_opt if accuracy_score(y_test, xgb_pred_opt) >= accuracy_score(y_test, xgb_pred_default)

rf_pred_default = (rf_test_prob >= 0.5).astype(int)
rf_pred_opt = (rf_test_prob >= rf_opt_thresh).astype(int)
rf_best_pred = rf_pred_opt if accuracy_score(y_test, rf_pred_opt) >= accuracy_score(y_test, rf_pred_default)

# Pairwise McNemar's tests
print("McNemar's Test Results (using best threshold per model):")
print("="*60)

pairs = [
    ("NGBost vs XGBoost", ngb_best_pred, xgb_best_pred),
```

```

        ("NGBoost vs RF", ngb_best_pred, rf_best_pred),
        ("XGBoost vs RF", xgb_best_pred, rf_best_pred),
    ]

    for name, pred1, pred2 in pairs:
        stat, p_val = McNemar_test(y_test, pred1, pred2)
        significance = "Significant" if p_val < 0.05 else "Not Significant"
        print(f" {name}: statistic={stat:.4f}, p-value={p_val:.4f} ({significance})")

```

McNemar's Test Results (using best threshold per model):

```

=====
NGBoost vs XGBoost: statistic=3.3220, p-value=0.0684 (Not Significant)
NGBoost vs RF: statistic=2.0167, p-value=0.1556 (Not Significant)
XGBoost vs RF: statistic=0.1143, p-value=0.7353 (Not Significant)

```

✓ Cell 13: Uncertainty Zone Analysis

Analisis 5 zona probabilitas untuk setiap model.

```

def uncertainty_zone_analysis(y_true, y_prob, model_name):
    """Analyze prediction uncertainty in 5 probability zones."""
    zones = [
        ("High Confidence (Not Potable)", 0.0, 0.2),
        ("Moderate (Not Potable)", 0.2, 0.4),
        ("Uncertain", 0.4, 0.6),
        ("Moderate (Potable)", 0.6, 0.8),
        ("High Confidence (Potable)", 0.8, 1.0),
    ]

    print(f"\n{model_name} - Uncertainty Zone Analysis:")
    print(f"{'Zone':<35} {'Count':<8} {'% Total':<10} {'Actual % Potable':<18}")
    print("-" * 75)

    for zone_name, low, high in zones:
        mask = (y_prob >= low) & (y_prob < high)
        count = mask.sum()
        pct = count / len(y_prob) * 100
        if count > 0:
            actual_pct = y_true[mask].mean() * 100
        else:
            actual_pct = 0
        print(f" {zone_name:<33} {count:<8} {pct:<10.1f} {actual_pct:<18.1f}")

    print("\n"*80)
    print("UNCERTAINTY ZONE ANALYSIS (Test Set)")
    print("\n"*80)

    uncertainty_zone_analysis(y_test, ngb_test_prob, "NGBoost")
    uncertainty_zone_analysis(y_test, xgb_test_prob, "XGBoost")
    uncertainty_zone_analysis(y_test, rf_test_prob, "Random Forest")

```

=====

UNCERTAINTY ZONE ANALYSIS (Test Set)

=====

NGBoost - Uncertainty Zone Analysis:

Zone	Count	% Total	Actual % Potable
High Confidence (Not Potable)	169	34.3	26.6
Moderate (Not Potable)	156	31.7	37.8
Uncertain	112	22.8	45.5
Moderate (Potable)	31	6.3	51.6
High Confidence (Potable)	24	4.9	87.5

XGBoost - Uncertainty Zone Analysis:

Zone	Count	% Total	Actual % Potable
High Confidence (Not Potable)	34	6.9	17.6
Moderate (Not Potable)	269	54.7	30.5
Uncertain	144	29.3	45.1
Moderate (Potable)	39	7.9	84.6
High Confidence (Potable)	6	1.2	100.0

Random Forest - Uncertainty Zone Analysis:

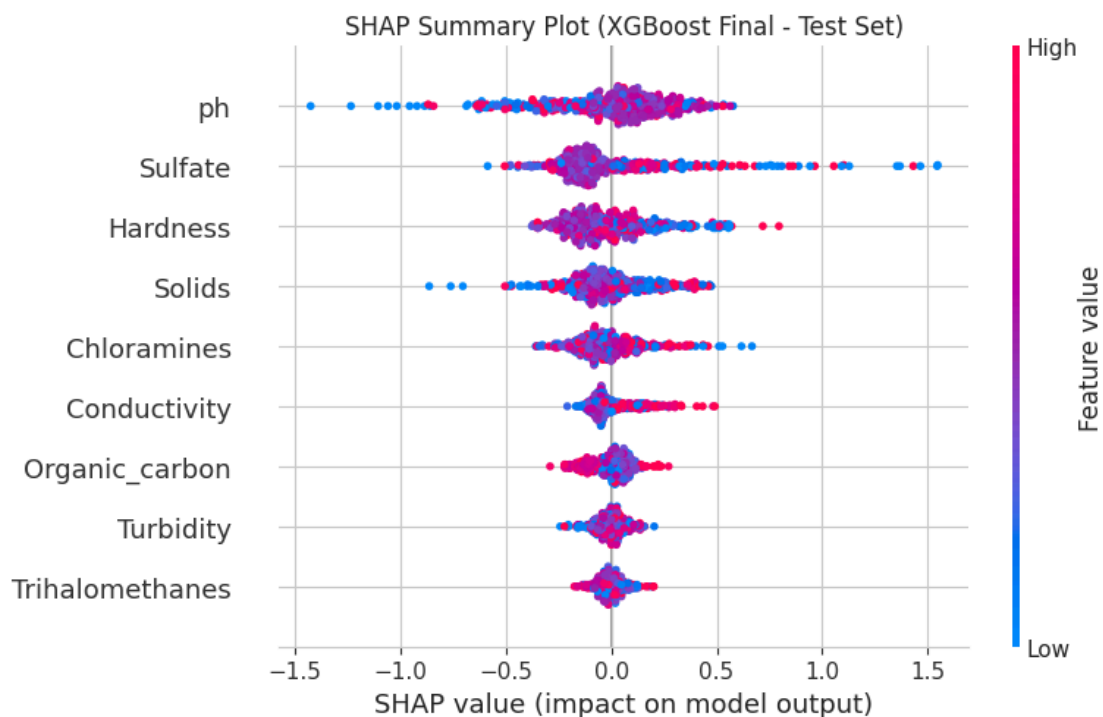
Zone	Count	% Total	Actual % Potable
------	-------	---------	------------------

High Confidence (Not Potable)	19	3.9	21.1
Moderate (Not Potable)	287	58.3	29.6
Uncertain	152	30.9	47.4
Moderate (Potable)	29	5.9	89.7
High Confidence (Potable)	5	1.0	100.0

✓ Cell 14: Visualisasi - SHAP Summary Plot (Final NGBoost)

```
# SHAP analysis on final NGBoost model
# NGBoost uses DecisionTreeRegressor as base learner, use KernelExplainer or TreeExplainer on XGBoost
# For consistency, we use XGBoost final model for SHAP (more interpretable)
explainer_final = shap.TreeExplainer(xgb_final)
shap_values_final = explainer_final.shap_values(X_test_scaled)

plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values_final, X_test_scaled, feature_names=feature_names, show=False)
plt.title("SHAP Summary Plot (XGBoost Final - Test Set)")
plt.tight_layout()
plt.savefig("figures/shap_summary_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```



✓ Cell 15: Visualisasi - Threshold Optimization Plot

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

thresholds = np.arange(0.30, 0.70, 0.01)

for ax, (name, val_prob, opt_thresh) in zip(axes, [
    ("NGBoost", ngb_val_prob, ngb_opt_thresh),
    ("XGBoost", xgb_val_prob, xgb_opt_thresh),
    ("Random Forest", rf_val_prob, rf_opt_thresh)
]):
    f1_scores = []
    acc_scores = []
    for t in thresholds:
        y_pred = (val_prob >= t).astype(int)
        f1_scores.append(f1_score(y_val, y_pred, zero_division=0))
        acc_scores.append(accuracy_score(y_val, y_pred))

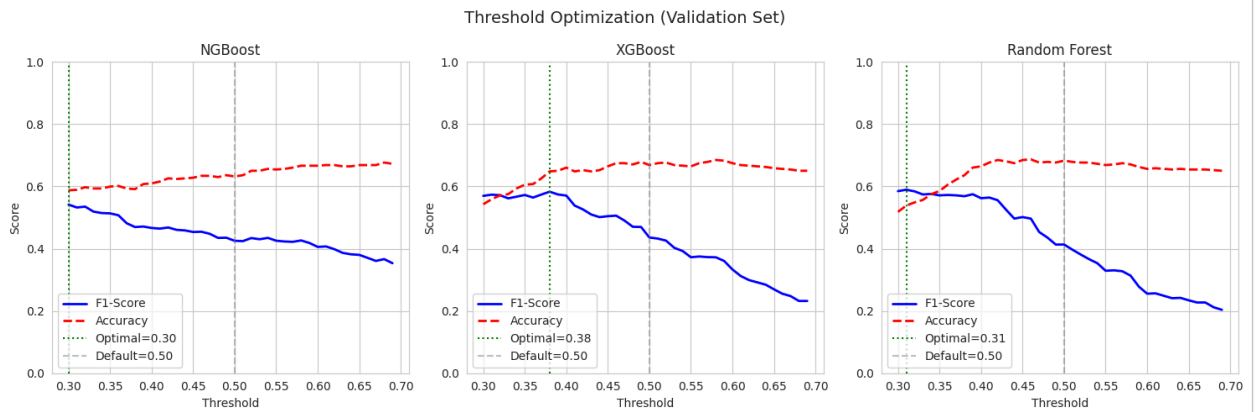
    ax.plot(thresholds, f1_scores, "b-", label="F1-Score", linewidth=2)
    ax.plot(thresholds, acc_scores, "r--", label="Accuracy", linewidth=2)
    ax.axvline(x=opt_thresh, color="green", linestyle=":", label=f"Optimal={opt_thresh:.2f}")
```

```

ax.axvline(x=0.5, color="gray", linestyle="--", alpha=0.5, label="Default=0.50")
ax.set_xlabel("Threshold")
ax.set_ylabel("Score")
ax.set_title(f"{name}")
ax.legend(loc="lower left")
ax.set_ylim([0, 1])

plt.suptitle("Threshold Optimization (Validation Set)", fontsize=14)
plt.tight_layout()
plt.savefig("figures/threshold_optimization_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()

```



✓ Cell 16: Visualisasi - Calibration Curves

```

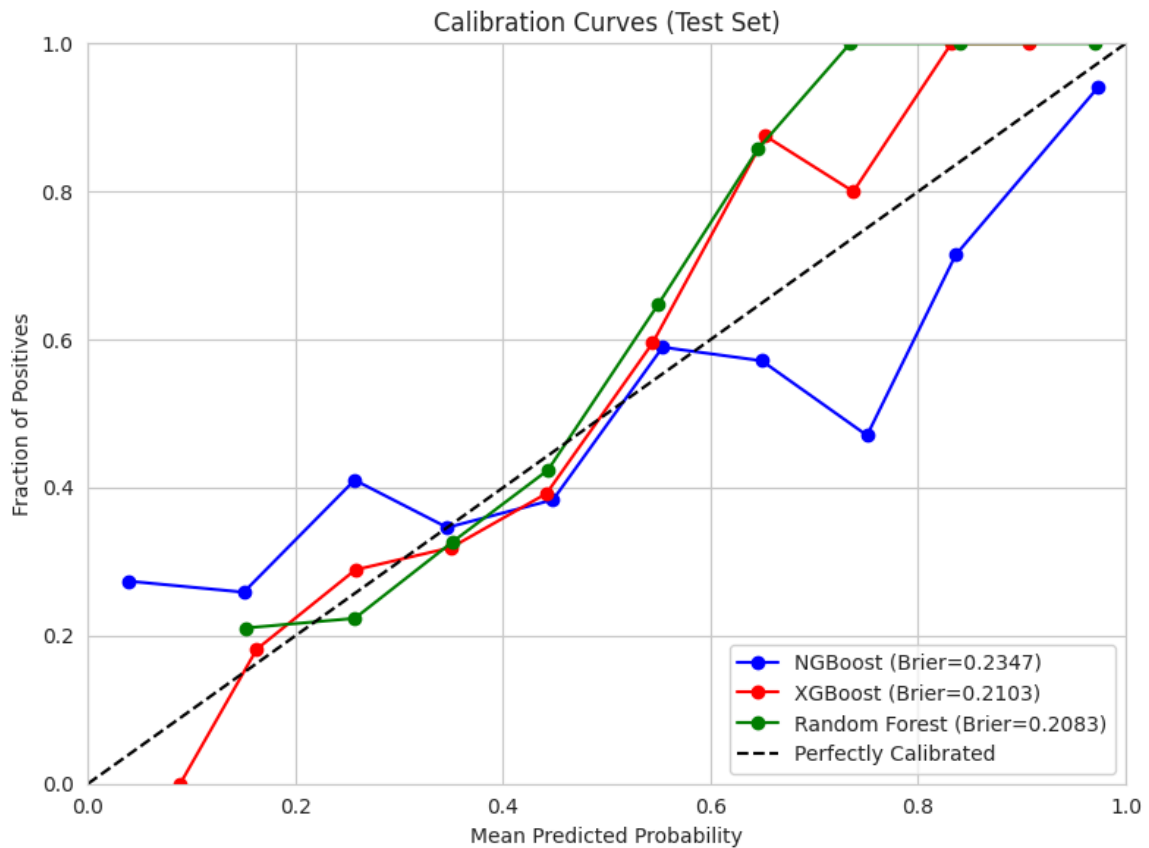
fig, ax = plt.subplots(1, 1, figsize=(8, 6))

for name, prob, color in [("NGBoost", ngb_test_prob, "blue"),
                          ("XGBoost", xgb_test_prob, "red"),
                          ("Random Forest", rf_test_prob, "green")]:
    fraction_positive, mean_predicted = calibration_curve(y_test, prob, n_bins=10)
    brier = brier_score_loss(y_test, prob)
    ax.plot(mean_predicted, fraction_positive, "o-", color=color,
            label=f"{name} (Brier={brier:.4f})")

ax.plot([0, 1], [0, 1], "k--", label="Perfectly Calibrated")
ax.set_xlabel("Mean Predicted Probability")
ax.set_ylabel("Fraction of Positives")
ax.set_title("Calibration Curves (Test Set)")
ax.legend(loc="lower right")
ax.set_xlim([0, 1])
ax.set_ylim([0, 1])

plt.tight_layout()
plt.savefig("figures/calibration_curves_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()

```



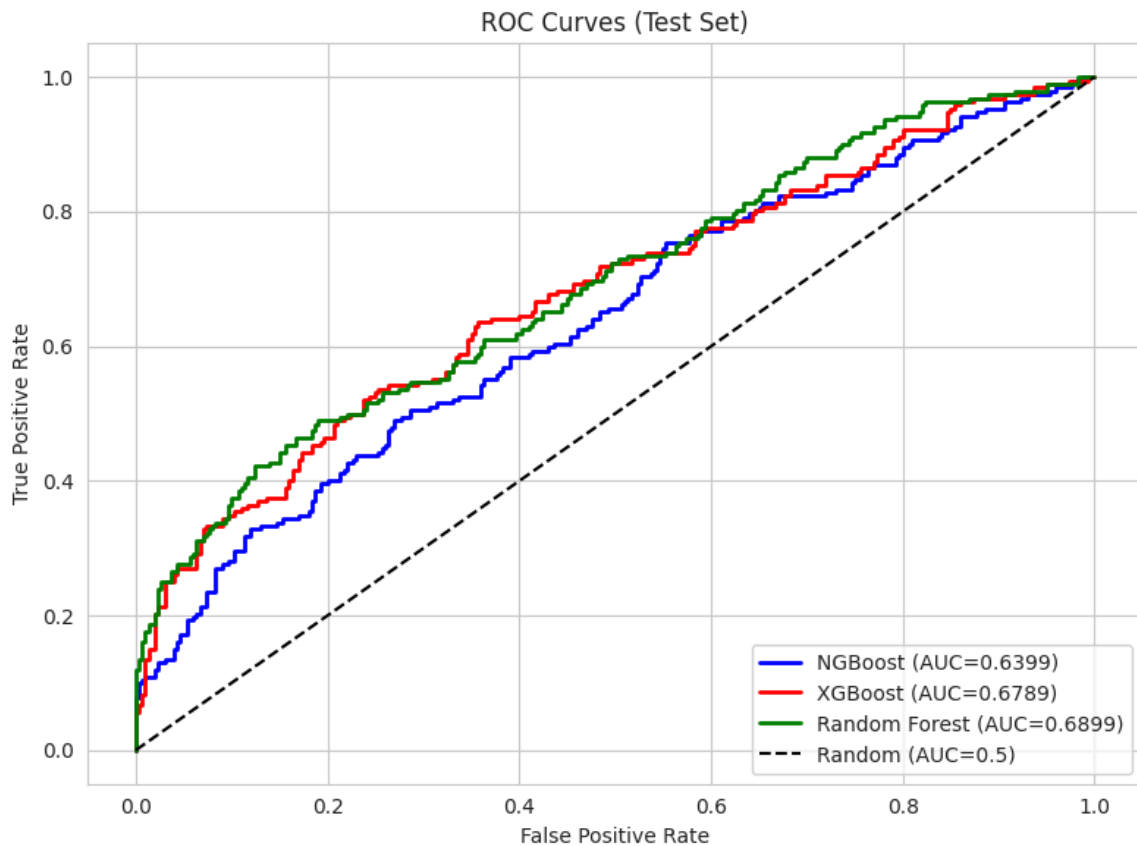
Cell 17: Visualisasi - ROC Curves

```
fig, ax = plt.subplots(1, 1, figsize=(8, 6))

for name, prob, color in [("NGBoost", ngb_test_prob, "blue"),
                          ("XGBoost", xgb_test_prob, "red"),
                          ("Random Forest", rf_test_prob, "green")]:
    fpr, tpr, _ = roc_curve(y_test, prob)
    auc = roc_auc_score(y_test, prob)
    ax.plot(fpr, tpr, color=color, linewidth=2, label=f"{name} (AUC={auc:.4f})")

ax.plot([0, 1], [0, 1], "k--", label="Random (AUC=0.5)")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_title("ROC Curves (Test Set)")
ax.legend(loc="lower right")

plt.tight_layout()
plt.savefig("figures/roc_curves_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```

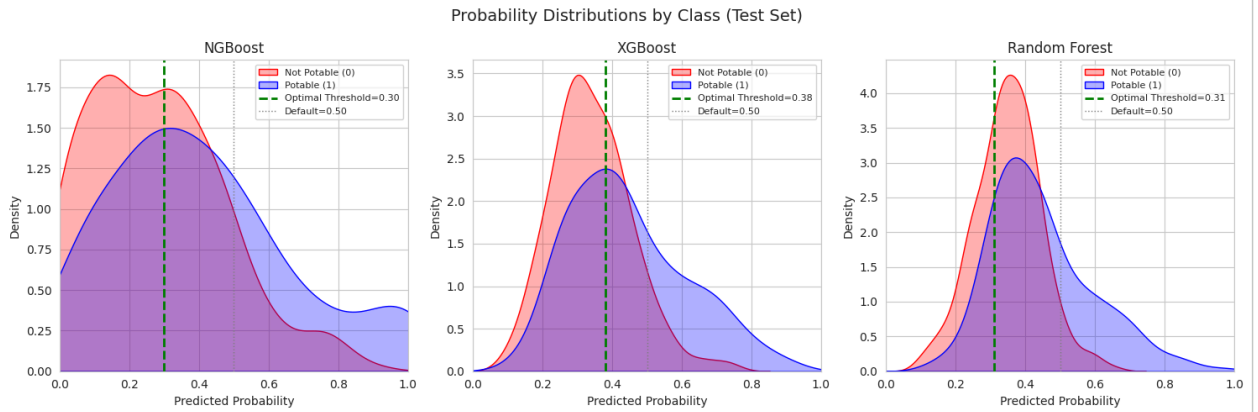


Cell 18: Visualisasi - KDE Distributions

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for ax, (name, prob, opt_thresh) in zip(axes, [
    ("NGBoost", ngb_test_prob, ngb_opt_thresh),
    ("XGBoost", xgb_test_prob, xgb_opt_thresh),
    ("Random Forest", rf_test_prob, rf_opt_thresh)
]):
    # KDE for each class
    sns.kdeplot(prob[y_test == 0], ax=ax, color="red", label="Not Potable (0)", fill=True, alpha=0.3)
    sns.kdeplot(prob[y_test == 1], ax=ax, color="blue", label="Potable (1)", fill=True, alpha=0.3)
    ax.axvline(x=opt_thresh, color="green", linestyle="--", linewidth=2, label=f"Optimal Threshold={opt_thresh}")
    ax.axvline(x=0.5, color="gray", linestyle=":", linewidth=1, label="Default=0.50")
    ax.set_xlabel("Predicted Probability")
    ax.set_ylabel("Density")
    ax.set_title(f"{name}")
    ax.legend(fontsize=8)
    ax.set_xlim([0, 1])

plt.suptitle("Probability Distributions by Class (Test Set)", fontsize=14)
plt.tight_layout()
plt.savefig("figures/kde_distributions_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```



✓ Cell 19: Visualisasi - Confusion Matrices

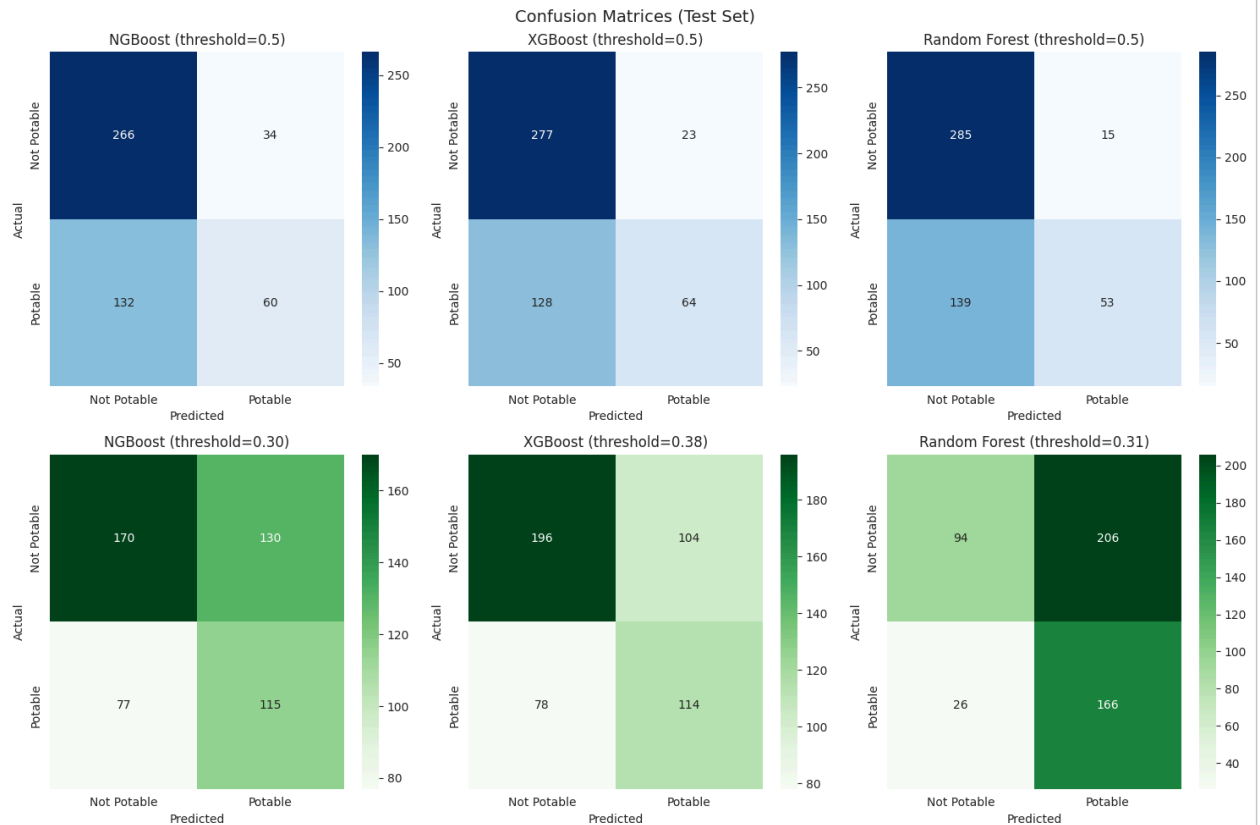
```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

models_data = [
    ("NGBoost", ngb_test_prob, ngb_opt_thresh),
    ("XGBoost", xgb_test_prob, xgb_opt_thresh),
    ("Random Forest", rf_test_prob, rf_opt_thresh)
]

# Top row: default threshold (0.5)
for idx, (name, prob, opt_thresh) in enumerate(models_data):
    y_pred = (prob >= 0.5).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=axes[0, idx],
                xticklabels=["Not Potable", "Potable"],
                yticklabels=["Not Potable", "Potable"])
    axes[0, idx].set_title(f"{name} (threshold=0.5)")
    axes[0, idx].set_xlabel("Predicted")
    axes[0, idx].set_ylabel("Actual")

# Bottom row: optimal threshold
for idx, (name, prob, opt_thresh) in enumerate(models_data):
    y_pred = (prob >= opt_thresh).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Greens", ax=axes[1, idx],
                xticklabels=["Not Potable", "Potable"],
                yticklabels=["Not Potable", "Potable"])
    axes[1, idx].set_title(f"{name} (threshold={opt_thresh:.2f})")
    axes[1, idx].set_xlabel("Predicted")
    axes[1, idx].set_ylabel("Actual")

plt.suptitle("Confusion Matrices (Test Set)", fontsize=14)
plt.tight_layout()
plt.savefig("figures/confusion_matrices_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```



Cell 20: Visualisasi - Feature Importance (Tuned Models)

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

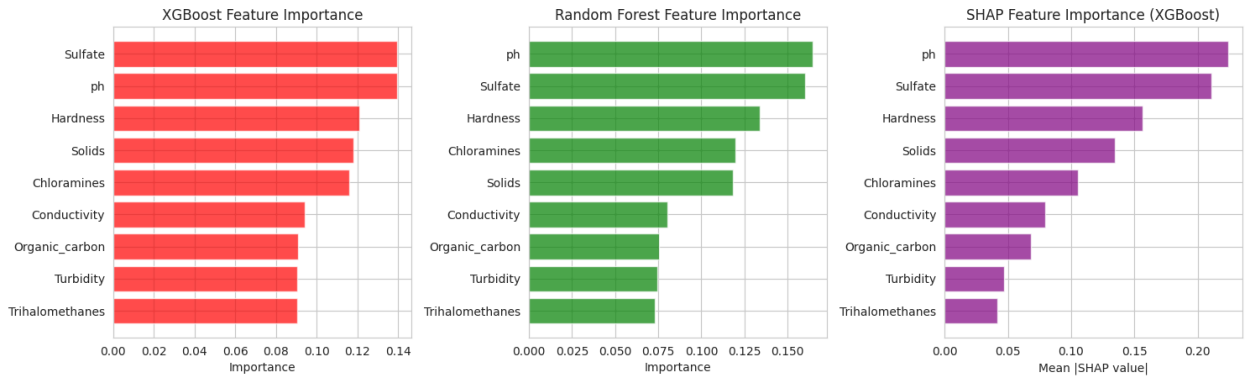
# XGBoost feature importance
xgb_importance = xgb_final.feature_importances_
sorted_idx = np.argsort(xgb_importance)
axes[0].barh(range(len(feature_names)), xgb_importance[sorted_idx], color="red", alpha=0.7)
axes[0].set_yticks(range(len(feature_names)))
axes[0].set_yticklabels([feature_names[i] for i in sorted_idx])
axes[0].set_title("XGBoost Feature Importance")
axes[0].set_xlabel("Importance")

# Random Forest feature importance
rf_importance = rf_final.feature_importances_
sorted_idx_rf = np.argsort(rf_importance)
axes[1].barh(range(len(feature_names)), rf_importance[sorted_idx_rf], color="green", alpha=0.7)
axes[1].set_yticks(range(len(feature_names)))
axes[1].set_yticklabels([feature_names[i] for i in sorted_idx_rf])
axes[1].set_title("Random Forest Feature Importance")
axes[1].set_xlabel("Importance")

# SHAP-based importance (from XGBoost)
shap_importance = np.abs(shap_values_final).mean(axis=0)
sorted_idx_shap = np.argsort(shap_importance)
axes[2].barh(range(len(feature_names)), shap_importance[sorted_idx_shap], color="purple", alpha=0.7)
axes[2].set_yticks(range(len(feature_names)))
axes[2].set_yticklabels([feature_names[i] for i in sorted_idx_shap])
axes[2].set_title("SHAP Feature Importance (XGBoost)")
axes[2].set_xlabel("Mean |SHAP value|")

plt.suptitle("Feature Importance Comparison (Tuned Models)", fontsize=14)
plt.tight_layout()
plt.savefig("figures/feature_importance_v4.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```

Feature Importance Comparison (Tuned Models)



✓ Cell 21: Stratified 5-Fold Cross-Validation (Full Dataset)

Evaluasi pada seluruh 3276 sampel. Per fold:

1. Impute + Scale pada train fold
2. Train dengan best hyperparameters (TANPA SMOTE)
3. Threshold optimize pada inner validation
4. Evaluate pada test fold

```
# Stratified 5-Fold CV on full dataset
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results = {"NGBoost": [], "XGBoost": [], "RF": []}
cv_f1_results = {"NGBoost": [], "XGBoost": [], "RF": []}
cv_auc_results = {"NGBoost": [], "XGBoost": [], "RF": []}

print("Stratified 5-Fold Cross-Validation")
print("="*60)

for fold_idx, (train_idx, test_idx) in enumerate(skf.split(X_imputed, y)):
    print(f"\nFold {fold_idx + 1}/5...")

    X_fold_train = X_imputed.iloc[train_idx]
    y_fold_train = y.iloc[train_idx]
    X_fold_test = X_imputed.iloc[test_idx]
    y_fold_test = y.iloc[test_idx]

    # Inner split for threshold optimization
    X_fold_tr, X_fold_val, y_fold_tr, y_fold_val = train_test_split(
        X_fold_train, y_fold_train, test_size=0.15, stratify=y_fold_train, random_state=42
    )

    # Impute (fit on fold train)
    fold_imputer = SimpleImputer(strategy="median")
    X_fold_tr_imp = fold_imputer.fit_transform(X_fold_tr)
    X_fold_val_imp = fold_imputer.transform(X_fold_val)
    X_fold_test_imp = fold_imputer.transform(X_fold_test)

    # Scale (fit on fold train)
    fold_scaler = MinMaxScaler()
    X_fold_tr_sc = fold_scaler.fit_transform(X_fold_tr_imp)
    X_fold_val_sc = fold_scaler.transform(X_fold_val_imp)
    X_fold_test_sc = fold_scaler.transform(X_fold_test_imp)

    # Train models with best hyperparameters
    # NGBoost
    ngb_cv = NGBClassifier(
        Dist=Bernoulli,
        n_estimators=best_ngb_params["n_estimators"],
        learning_rate=best_ngb_params["learning_rate"],
        minibatch_frac=best_ngb_params["minibatch_frac"],
        col_sample=0.8,
        random_state=42,
        verbose=False
```

```

)
ngb_cv.fit(X_fold_tr_sc, y_fold_tr)

# XGBoost
xgb_cv = XGBClassifier(
    **xgb_search.best_params_,
    eval_metric="logloss", use_label_encoder=False, verbosity=0, random_state=42
)
xgb_cv.fit(X_fold_tr_sc, y_fold_tr)

# Random Forest
rf_cv = RandomForestClassifier(**rf_search.best_params_, random_state=42)
rf_cv.fit(X_fold_tr_sc, y_fold_tr)

# Threshold optimization on inner validation
ngb_val_p = ngb_cv.predict_proba(X_fold_val_sc)[: , 1]
xgb_val_p = xgb_cv.predict_proba(X_fold_val_sc)[: , 1]
rf_val_p = rf_cv.predict_proba(X_fold_val_sc)[: , 1]

ngb_t, _ = find_optimal_threshold(y_fold_val, ngb_val_p)
xgb_t, _ = find_optimal_threshold(y_fold_val, xgb_val_p)
rf_t, _ = find_optimal_threshold(y_fold_val, rf_val_p)

# Evaluate on test fold
ngb_test_p = ngb_cv.predict_proba(X_fold_test_sc)[: , 1]
xgb_test_p = xgb_cv.predict_proba(X_fold_test_sc)[: , 1]
rf_test_p = rf_cv.predict_proba(X_fold_test_sc)[: , 1]

ngb_pred_cv = (ngb_test_p >= ngb_t).astype(int)
xgb_pred_cv = (xgb_test_p >= xgb_t).astype(int)
rf_pred_cv = (rf_test_p >= rf_t).astype(int)

cv_results["NGBoost"].append(accuracy_score(y_fold_test, ngb_pred_cv))
cv_results["XGBoost"].append(accuracy_score(y_fold_test, xgb_pred_cv))
cv_results["RF"].append(accuracy_score(y_fold_test, rf_pred_cv))

cv_f1_results["NGBoost"].append(f1_score(y_fold_test, ngb_pred_cv, zero_division=0))
cv_f1_results["XGBoost"].append(f1_score(y_fold_test, xgb_pred_cv, zero_division=0))
cv_f1_results["RF"].append(f1_score(y_fold_test, rf_pred_cv, zero_division=0))

cv_auc_results["NGBoost"].append(roc_auc_score(y_fold_test, ngb_test_p))
cv_auc_results["XGBoost"].append(roc_auc_score(y_fold_test, xgb_test_p))
cv_auc_results["RF"].append(roc_auc_score(y_fold_test, rf_test_p))

print(f" NGBoost: Acc={cv_results["NGBoost"][-1]:.4f}, F1={cv_f1_results["NGBoost"][-1]:.4f}, AUC={cv_auc_results["NGBoost"][-1]:.4f}")
print(f" XGBoost: Acc={cv_results["XGBoost"][-1]:.4f}, F1={cv_f1_results["XGBoost"][-1]:.4f}, AUC={cv_auc_results["XGBoost"][-1]:.4f}")
print(f" RF: Acc={cv_results["RF"][-1]:.4f}, F1={cv_f1_results["RF"][-1]:.4f}, AUC={cv_auc_results["RF"][-1]:.4f}")

print("\n" + "="*60)
print("5-Fold CV Summary:")
print("="*60)
for model in ["NGBoost", "XGBoost", "RF"]:
    acc_mean = np.mean(cv_results[model])
    acc_std = np.std(cv_results[model])
    f1_mean = np.mean(cv_f1_results[model])
    f1_std = np.std(cv_f1_results[model])
    auc_mean = np.mean(cv_auc_results[model])
    auc_std = np.std(cv_auc_results[model])
    print(f" {model:8s}: Acc={acc_mean:.4f}+/-{acc_std:.4f}, F1={f1_mean:.4f}+/-{f1_std:.4f}, AUC={auc_mean:.4f}+/-{auc_std:.4f}")

```

Stratified 5-Fold Cross-Validation

=====

Fold 1/5...

```

NGBoost: Acc=0.6204, F1=0.5221, AUC=0.6439 (threshold=0.37)
XGBoost: Acc=0.5229, F1=0.5741, AUC=0.6789 (threshold=0.30)
RF:      Acc=0.5976, F1=0.5783, AUC=0.6872 (threshold=0.36)

```

Fold 2/5...

```

NGBoost: Acc=0.5939, F1=0.5317, AUC=0.6509 (threshold=0.30)
XGBoost: Acc=0.5939, F1=0.5994, AUC=0.7104 (threshold=0.31)
RF:      Acc=0.6321, F1=0.6144, AUC=0.7239 (threshold=0.34)

```

Fold 3/5...

```

NGBoost: Acc=0.5893, F1=0.5239, AUC=0.6491 (threshold=0.30)
XGBoost: Acc=0.6183, F1=0.6019, AUC=0.6973 (threshold=0.34)

```

```

RF:      Acc=0.6153, F1=0.5729, AUC=0.7014 (threshold=0.36)

Fold 4/5...
  NGBoost: Acc=0.5954, F1=0.5120, AUC=0.6160 (threshold=0.31)
  XGBoost: Acc=0.6046, F1=0.5775, AUC=0.6622 (threshold=0.33)
  RF:      Acc=0.5145, F1=0.5737, AUC=0.6706 (threshold=0.30)

Fold 5/5...
  NGBoost: Acc=0.5969, F1=0.5368, AUC=0.6343 (threshold=0.30)
  XGBoost: Acc=0.5969, F1=0.5913, AUC=0.6931 (threshold=0.32)
  RF:      Acc=0.5863, F1=0.5925, AUC=0.7018 (threshold=0.33)

=====
5-Fold CV Summary:
=====
  NGBoost : Acc=0.5992+/-0.0109, F1=0.5253+/-0.0085, AUC=0.6388+/-0.0128
  XGBoost : Acc=0.5873+/-0.0333, F1=0.5889+/-0.0113, AUC=0.6884+/-0.0165
  RF      : Acc=0.5891+/-0.0404, F1=0.5864+/-0.0157, AUC=0.6970+/-0.0177

```

✓ Cell 22: Perbandingan Semua Versi

Membandingkan hasil V1, V2, V3 (hardcoded) dengan V4 (computed).

```

# Hardcoded results from previous versions
comparison_data = {
    "Config": ["V1", "V2", "V3", "V4"],
    "Preprocessing": [
        "MICE+Standard+NoSMOTE",
        "Median+MinMax+SMOTE",
        "V2+ThresholdOpt",
        "Median+MinMax+NoSMOTE+Tuning+ThreshOpt"
    ],
}

# Get V4 results (using optimal threshold)
ngb_pred_v4 = (ngb_test_prob >= ngb_opt_thresh).astype(int)
xgb_pred_v4 = (xgb_test_prob >= xgb_opt_thresh).astype(int)
rf_pred_v4 = (rf_test_prob >= rf_opt_thresh).astype(int)

v4_ngb_acc = accuracy_score(y_test, ngb_pred_v4)
v4_xgb_acc = accuracy_score(y_test, xgb_pred_v4)
v4_rf_acc = accuracy_score(y_test, rf_pred_v4)
v4_ngb_f1 = f1_score(y_test, ngb_pred_v4, zero_division=0)
v4_xgb_f1 = f1_score(y_test, xgb_pred_v4, zero_division=0)
v4_rf_f1 = f1_score(y_test, rf_pred_v4, zero_division=0)

comparison_data["NGBoost_Acc"] = [0.6707, 0.6301, 0.5224, v4_ngb_acc]
comparison_data["XGBoost_Acc"] = [0.6707, 0.6179, 0.4959, v4_xgb_acc]
comparison_data["RF_Acc"] = [0.6585, 0.6545, 0.4614, v4_rf_acc]
comparison_data["NGBoost_F1"] = [0.4000, 0.5357, 0.5624, v4_ngb_f1]
comparison_data["XGBoost_F1"] = [0.4452, 0.4946, 0.5603, v4_xgb_f1]
comparison_data["RF_F1"] = [0.4085, 0.5058, 0.5705, v4_rf_f1]

comparison_df = pd.DataFrame(comparison_data)

print("="*100)
print("PERBANDINGAN SEMUA VERSI PIPELINE")
print("="*100)
print(comparison_df.to_string(index=False))

print("Analisis:")
print("-"*60)
print(f"V4 NGBoost Accuracy: {v4_ngb_acc:.4f} (vs V1: 0.6707, V2: 0.6301, V3: 0.5224)")
print(f"V4 XGBoost Accuracy: {v4_xgb_acc:.4f} (vs V1: 0.6707, V2: 0.6179, V3: 0.4959)")
print(f"V4 RF Accuracy: {v4_rf_acc:.4f} (vs V1: 0.6585, V2: 0.6545, V3: 0.4614)")
print(f"V4 NGBoost F1: {v4_ngb_f1:.4f} (vs V1: 0.4000, V2: 0.5357, V3: 0.5624)")
print(f"V4 XGBoost F1: {v4_xgb_f1:.4f} (vs V1: 0.4452, V2: 0.4946, V3: 0.5603)")
print(f"V4 RF F1: {v4_rf_f1:.4f} (vs V1: 0.4085, V2: 0.5058, V3: 0.5705)")

```

=====

PERBANDINGAN SEMUA VERSI PIPELINE

=====

Config	Preprocessing	NGBoost_Acc	XGBoost_Acc	RF_Acc	NGBoost_F1	XGBoost_F1	RF_F1
V1	MICE+Standard+NoSMOTE	0.670700	0.670700	0.658500	0.400000	0.445200	0.4085

V2	Median+MinMax+SMOTE	0.630100	0.617900	0.654500	0.535700	0.494600	0.5058
V3	V2+ThresholdOpt	0.522400	0.495900	0.461400	0.562400	0.560300	0.5705
V4	Median+MinMax+NoSMOTE+Tuning+ThreshOpt	0.579268	0.630081	0.528455	0.526316	0.556098	0.5886
Analysis:							

V4	NGBoost Accuracy:	0.5793	(vs V1: 0.6707,	V2: 0.6301,	V3: 0.5224)		
V4	XGBoost Accuracy:	0.6301	(vs V1: 0.6707,	V2: 0.6179,	V3: 0.4959)		
V4	RF Accuracy:	0.5285	(vs V1: 0.6585,	V2: 0.6545,	V3: 0.4614)		
V4	NGBoost F1:	0.5263	(vs V1: 0.4000,	V2: 0.5357,	V3: 0.5624)		
V4	XGBoost F1:	0.5561	(vs V1: 0.4452,	V2: 0.4946,	V3: 0.5603)		
V4	RF F1:	0.5887	(vs V1: 0.4085,	V2: 0.5058,	V3: 0.5705)		

- Cell 23: Ringkasan Hasil Final